**SIMATS ENGINEERING**
**CO5 ASSESSMENT TOOL 1**

**NAME: SRAVANTHI K**
**REG NO: 192372127**
**COURSE NAME: INTERNET PROGRAMMING**
**COURSE CODE: CSA4301**

1.A hospital management system requires a **SOAP-based web service** to manage patient information such as registration, appointment scheduling, and medical records. The system must ensure secure, structured, and interoperable communication between different hospital departments.

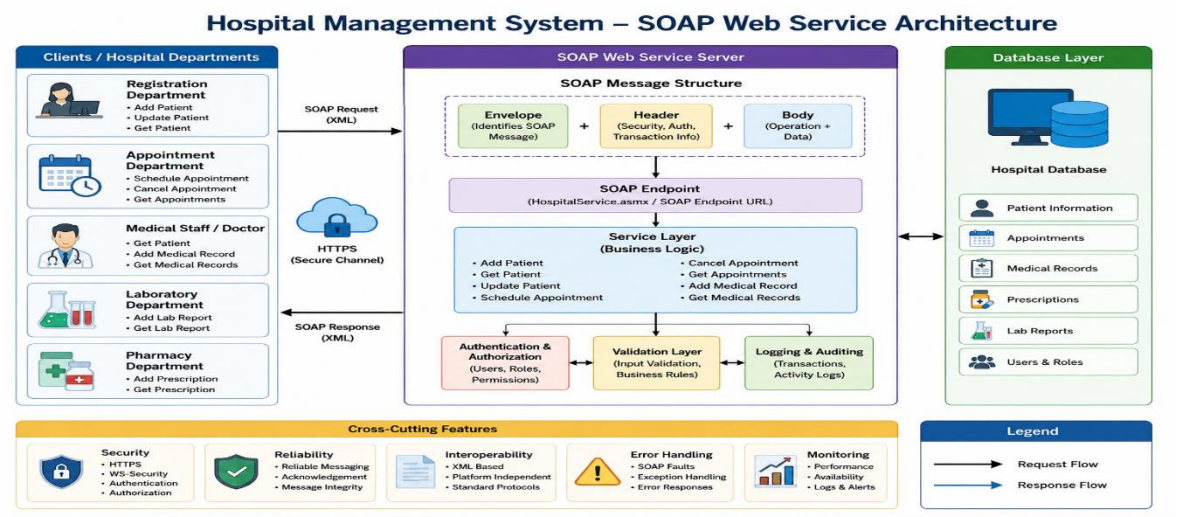
Design a **SOAP-based web service** for the hospital system. Explain the structure of SOAP messages (Envelope, Header, Body), define the operations involved (e.g., add patient, get records), and illustrate how data is exchanged between client and server. Justify the use of SOAP in this scenario considering reliability and security.

## 1. SOAP-Based Web Service for Hospital Management System

A **SOAP (Simple Object Access Protocol)** based web service can be designed to allow different hospital departments such as **Registration, Outpatient, Laboratory, Pharmacy, and Medical Records** to securely exchange patient information. SOAP is suitable because it provides a standardized XML-based communication mechanism with support for security, reliability, and interoperability.

## 1. System Architecture

The hospital management system can follow a **client-server architecture**:




Each department acts as a **SOAP client** and communicates with the hospital SOAP server using XML messages, normally over **HTTPS**.

**2. Main Operations**

The web service can provide the following operations:

| **Operation** | **Description** |
|---|---|
| addPatient() | Registers a new patient |
| getPatient() | Retrieves patient details |
| updatePatient() | Updates patient information |
| scheduleAppointment() | Schedules an appointment |
| cancelAppointment() | Cancels an appointment |
| getAppointments() | Retrieves appointment details |
| addMedicalRecord() | Adds a medical record |
| getMedicalRecords() | Retrieves a patient's medical records |

For example, the addPatient() operation can accept:

Patient ID

Patient Name

Age

Gender

Address

Phone Number

Blood Group

The server validates the information and stores it in the hospital database.

**3. Structure of a SOAP Message**

A SOAP message consists mainly of three important parts:

SOAP Message

The **Envelope** is the root element of every SOAP message. It identifies the XML document as a SOAP message.

**B. SOAP Header**

The **Header** contains optional information such as:

- Authentication details
- Authorization information
- Security tokens
- Transaction information
- Message identification
- Reliability information

For a hospital system, authentication information can be placed in the header.

**C. SOAP Body**

The **Body** contains the actual request or response. For example, it can contain patient registration information or medical records.

**D. SOAP Fault**

If an error occurs, the SOAP server can return a **Fault** message containing information about the error.

**4. SOAP Request – Add Patient**

Suppose the registration department wants to add a new patient.

The client sends a SOAP request such as:

<?xml version="1.0" encoding="UTF-8"?>

```
<soap:Envelope
    xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:h="http://hospital.example.com/service">

    <soap:Header>
        <h:Authentication>
            <h:Username>registration_user</h:Username>
            <h:Token>ABC123XYZ</h:Token>
        </h:Authentication>
    </soap:Header>

    <soap:Body>
        <h:AddPatient>
            <h:Patient>
                <h:PatientID>P1001</h:PatientID>
                <h:Name>Rahul Kumar</h:Name>
                <h:Age>35</h:Age>
                <h:Gender>Male</h:Gender>
                <h:BloodGroup>O+</h:BloodGroup>
                <h:Phone>9876543210</h:Phone>
            </h:Patient>
        </h:AddPatient>
    </soap:Body>
```

```
</soap:Envelope>
```

**5. SOAP Response**

After receiving the request, the SOAP server authenticates the user, validates the data, stores it in the database, and returns a response.

```
<?xml version="1.0" encoding="UTF-8"?>

<soap:Envelope

   xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"

   xmlns:h="http://hospital.example.com/service">

   <soap:Header>

      <h:ResponseInfo>

         <h:TransactionID>TXN10001</h:TransactionID>

      </h:ResponseInfo>

   </soap:Header>



   <soap:Body>

      <h:AddPatientResponse>

         <h:Status>SUCCESS</h:Status>

         <h:Message>Patient registered successfully</h:Message>

         <h:PatientID>P1001</h:PatientID>

      </h:AddPatientResponse>

   </soap:Body>

</soap:Envelope>
```

The client receives confirmation that the patient has been successfully registered.

**6. Get Medical Records Operation**

A doctor or authorized department may request a patient's medical records.

**Request**

```
<soap:Envelope
   xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
   xmlns:h="http://hospital.example.com/service">
   <soap:Header>
      <h:Authentication>
         <h:Username>doctor01</h:Username>
         <h:Token>SECURETOKEN123</h:Token>
      </h:Authentication>
   </soap:Header>
   <soap:Body>
      <h:GetMedicalRecords>
         <h:PatientID>P1001</h:PatientID>
      </h:GetMedicalRecords>
   </soap:Body>

</soap:Envelope>
```

**Response**

```
<soap:Envelope
   xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
   xmlns:h="http://hospital.example.com/service">
```

```
    <soap:Body>
        <h:GetMedicalRecordsResponse>
            <h:PatientID>P1001</h:PatientID>
            <h:MedicalRecord>
                <h:VisitDate>2026-08-20</h:VisitDate>
                <h:Doctor>Dr. Kumar</h:Doctor>
                <h:Diagnosis>Fever</h:Diagnosis>
                <h:Prescription>Medication prescribed</h:Prescription>
            </h:MedicalRecord>
        </h:GetMedicalRecordsResponse>
    </soap:Body>
</soap:Envelope>
```

Only an authenticated and authorized user should be allowed to retrieve such information.

**7. WSDL Definition**

A SOAP service can publish a **WSDL (Web Services Description Language)** document. WSDL describes how clients can communicate with the service.

A simplified WSDL structure could contain:

```
<definitions>
    <service name="HospitalService">
        <portType name="HospitalPortType">
            <operation name="AddPatient">
                ...
            </operation>
            <operation name="GetPatient">
```

```
            ...
        </operation>
        <operation name="ScheduleAppointment">
            ...
        </operation>
        <operation name="GetMedicalRecords">
            ...
        </operation>
    </portType>
  </service>
</definitions>
```

The WSDL specifies:

- Available operations
- Input parameters
- Output parameters
- Message formats
- Service endpoint
- Communication protocol

Therefore, different hospital departments can develop clients in different programming languages while still communicating with the same service.

**8. Security**

Security is extremely important because the system handles confidential medical information.

**a) HTTPS**

SOAP messages should be transmitted using **HTTPS** to encrypt communication between the client and server.

SOAP Client


SOAP Server

This prevents unauthorized parties from easily reading the data during transmission.

**b) Authentication**

The SOAP Header can contain authentication information such as a security token or credentials.

```
<soap:Header>
  <Authentication>
    <Username>doctor01</Username>
    <Token>SECURETOKEN123</Token>
  </Authentication>
</soap:Header>
```

**c) Authorization**

Authentication alone is not sufficient. The server should determine what each user is allowed to access.

For example:

Doctor → Patient records

Nurse → Limited patient information

Receptionist → Registration and appointments

Administrator → Authorized administrative information

**d) WS-Security**

SOAP supports **WS-Security**, which can provide message-level security features such as authentication, integrity, and encryption.

This is useful when sensitive medical information must be protected even beyond transport-level HTTPS.

**9. Reliability**

SOAP is suitable for hospital systems because reliability is important when exchanging critical information.

For example, an appointment request should not be accidentally lost or processed incorrectly.

SOAP-based web services can work with standards such as **WS-ReliableMessaging** to support reliable message delivery, acknowledgements, ordering, and handling of communication failures.

A reliable communication flow can be:

Client


Client

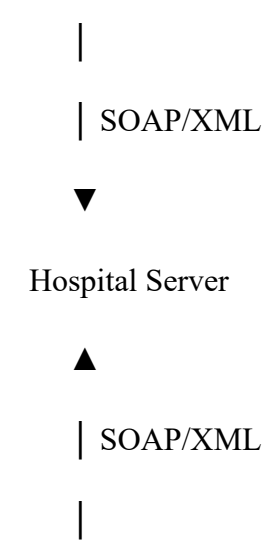
If an error occurs, the server can return a SOAP Fault:

```
<soap:Fault>
   <faultcode>soap:Server</faultcode>
   <faultstring>
      Patient record could not be retrieved
   </faultstring>
</soap:Fault>
```

**10. Interoperability**

SOAP uses **XML**, which is platform-independent.

For example:



.NET Medical Records System

Even if different departments use different technologies such as **Java, C#, Python, or PHP**, they can communicate through the same SOAP interface.

## 11. Advantages of SOAP for Hospital Management

1. **High interoperability** – Different hospital applications and departments can communicate regardless of programming language or operating system.
2. **Strong security support** – HTTPS and WS-Security can protect sensitive patient information.
3. **Reliable communication** – SOAP can be combined with reliable messaging standards for important transactions.
4. **Structured communication** – XML provides a well-defined structure for requests and responses.
5. **Standardized interface** – WSDL clearly defines available services and their input/output formats.
6. **Error handling** – SOAP Fault provides a standardized mechanism for reporting errors.
7. **Transaction support** – SOAP-based enterprise standards can support complex business transactions.

## 12. Conclusion

A **SOAP-based Hospital Management Web Service** provides a structured and standardized method for sharing patient registration, appointment, and medical-record information between hospital departments. The SOAP **Envelope** provides the overall message structure, the **Header** carries security and other control information, and the **Body** contains the actual operation and patient data.

Using **HTTPS, authentication, authorization, WS-Security, WSDL, and reliable messaging**, the system can provide secure and dependable communication. Therefore, SOAP is particularly appropriate for a hospital environment where **data security, reliability, interoperability, and standardized communication** are more important than simply minimizing message size.

**2.An enterprise application needs to expose its web services so that external systems can understand how to interact with them. The organization plans to use WSDL (Web Services Description Language) to describe the service.**

**Design the WSDL structure for a web service that provides operations such as retrieving and updating data. Explain the key components of WSDL (types, message, portType, binding, service), and illustrate how they define the service interface and communication details. Provide a clear structure and justify its importance in service integration**.

**WSDL Structure for a Web Service – 10 Marks**

**WSDL (Web Services Description Language)** is an XML-based language used to describe a web service, its operations, input/output data, communication protocol, and service location. It helps external systems understand and consume the service.

**1. Basic WSDL Structure**

WSDL Document

│

├── Types → Defines data types

├── Message → Defines input/output messages

├── PortType → Defines operations

├── Binding → Defines communication protocol

└── Service → Defines service endpoint/location

**2. Example WSDL**

Consider a service with two operations: **GetData** and **UpdateData**.

```
<definitions name="DataService"
        targetNamespace="http://example.com/data">

  <types>
    <xsd:schema>
```

```
        <xsd:element name="GetDataRequest">
            <xsd:complexType>
                <xsd:sequence>
                    <xsd:element name="id" type="xsd:int"/>
                </xsd:sequence>
            </xsd:complexType>
        </xsd:element>
    </xsd:schema>
</types>

<message name="GetDataRequest">
    <part name="parameters" element="tns:GetDataRequest"/>
</message>

<message name="GetDataResponse">
    <part name="result" type="xsd:string"/>
</message>

<portType name="DataPortType">
    <operation name="GetData">
        <input message="tns:GetDataRequest"/>
        <output message="tns:GetDataResponse"/>
    </operation>
```

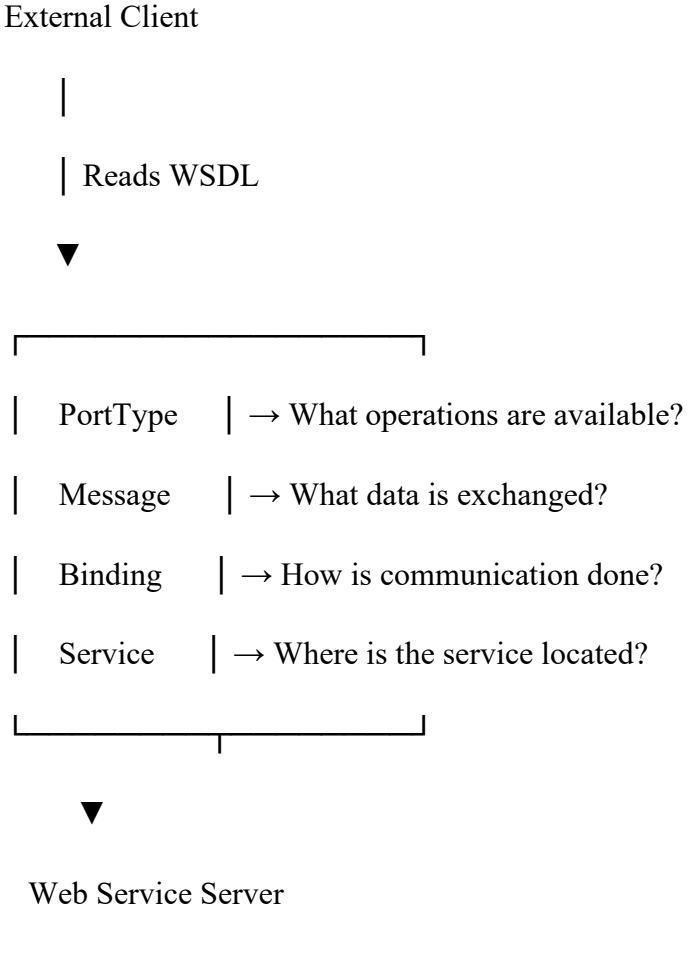
```
    <operation name="UpdateData">
      <input message="tns:UpdateDataRequest"/>
      <output message="tns:UpdateDataResponse"/>
    </operation>
  </portType>

  <binding name="DataBinding" type="tns:DataPortType">
    <soap:binding transport="http://schemas.xmlsoap.org/soap/http"/>
  </binding>

  <service name="DataService">
    <port name="DataPort"
        binding="tns:DataBinding">
      <soap:address location="https://example.com/DataService"/>
    </port>
  </service>

</definitions>
```

**3. Key Components**

| Component | Purpose |
|---|---|
| **Types** | Defines data types and structure using XML Schema (XSD). |

| Component | Purpose |
| --- | --- |
| **Message** | Defines the data exchanged between client and server. |
| **PortType** | Defines the available operations and their input/output messages. |
| **Binding** | Specifies how the service communicates, such as SOAP over HTTP. |
| **Service** | Specifies the actual endpoint URL where the service is available. |

**4. How It Works**




**5. Importance of WSDL**

- Provides a **standard description** of web services.

- Enables **different applications and platforms** to communicate.
- Clearly defines operations, input, and output.
- Supports automatic **client/proxy generation**.
- Separates the **service interface from implementation**.
- Simplifies integration of external systems.

**Conclusion:** WSDL acts as a **contract between the service provider and service consumer**, clearly describing **what the service does, how to communicate with it, and where to access it**. This makes enterprise service integration easier, standardized, and interoperable.

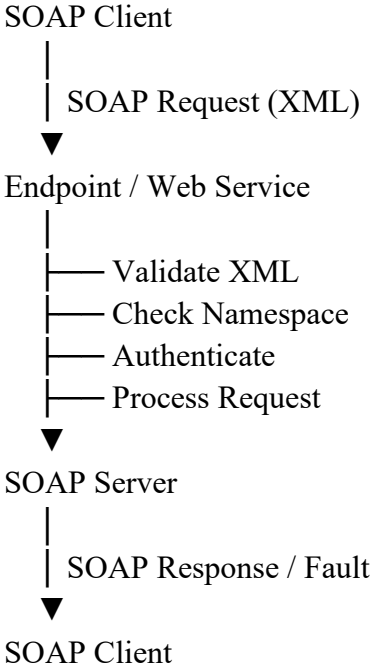
**3. A company is experiencing issues in its SOAP-based communication system where requests are not processed correctly, and responses are either delayed or incorrect.**

**Analyze the SOAP communication process and identify possible issues such as message formatting errors, incorrect namespaces, or endpoint problems. Propose a debugging approach to resolve these issues, including validation of SOAP messages and error handling techniques. Explain how the corrected system ensures reliable communication.**

**SOAP Communication Debugging – 10 Marks**

SOAP communication works through a **client-server request and response process**. If requests are not processed correctly or responses are delayed/incorrect, the SOAP message, namespace, endpoint, or server configuration may be the cause.

**1. SOAP Communication Process**



**2. Possible Problems**

| Problem | Effect |
|---|---|
| **Message formatting error** | Invalid XML or missing elements causes request failure |
| **Incorrect namespace** | Server cannot recognize the requested operation |
| **Wrong endpoint URL** | Request does not reach the correct service |
| **Incorrect SOAPAction** | Server may not identify the operation |
| **Invalid data types** | Request fails during validation |
| **Authentication failure** | Server rejects the request |
| **Network/server delay** | Response is slow or unavailable |
| **Incorrect response mapping** | Client receives unexpected data |

**3. Debugging Approach**

**Step 1 – Check Endpoint**

Verify that the SOAP request is being sent to the correct service URL.

Client → Correct SOAP Endpoint → Server

**Step 2 – Validate SOAP XML**

Check that the message contains the correct:

- Envelope
- Header
- Body
- Opening and closing tags
- Required elements

Example:

```
<soap:Envelope>
  <soap:Header>
    ...
  </soap:Header>
  <soap:Body>
    ...
  </soap:Body>
</soap:Envelope>
```

**Step 3 – Check Namespaces**

Ensure that the namespace in the request matches the namespace expected by the WSDL.

```
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:tns="http://example.com/service">
```

An incorrect namespace can cause an **operation not found** error.

**Step 4 – Verify WSDL**

Compare the request with the WSDL to verify:

- Operation name
- Input/output messages
- Data types
- Endpoint

- Binding

**Step 5 – Check SOAP Headers**

Verify authentication tokens, SOAPAction, security information, and other required headers.

**4. Error Handling**

SOAP provides a standard **SOAP Fault** mechanism.

```
<soap:Fault>
  <faultcode>soap:Server</faultcode>
  <faultstring>
    Invalid patient ID
  </faultstring>
</soap:Fault>
```

The client should log the fault and handle errors appropriately instead of treating the response as a successful transaction.

**5. Testing and Monitoring**

Tools such as **SoapUI** or similar API testing tools can be used to send requests and inspect responses. Server logs should also be checked for:

- Request timestamps
- Error messages
- Authentication failures
- Processing time
- Database errors

**6. Ensuring Reliable Communication**

After correction:

Client
↓
Validate SOAP Request
↓
Correct Endpoint
↓
Correct Namespace & Operation
↓
Server Processing
↓
SOAP Response / Fault
↓
Client

**Conclusion:** SOAP communication problems can be resolved by systematically checking the **XML structure, namespaces, WSDL, endpoint, headers, network connection, and server logs**. Proper SOAP Fault handling, validation, logging, and testing ensure that requests are processed correctly and reliable responses are returned.